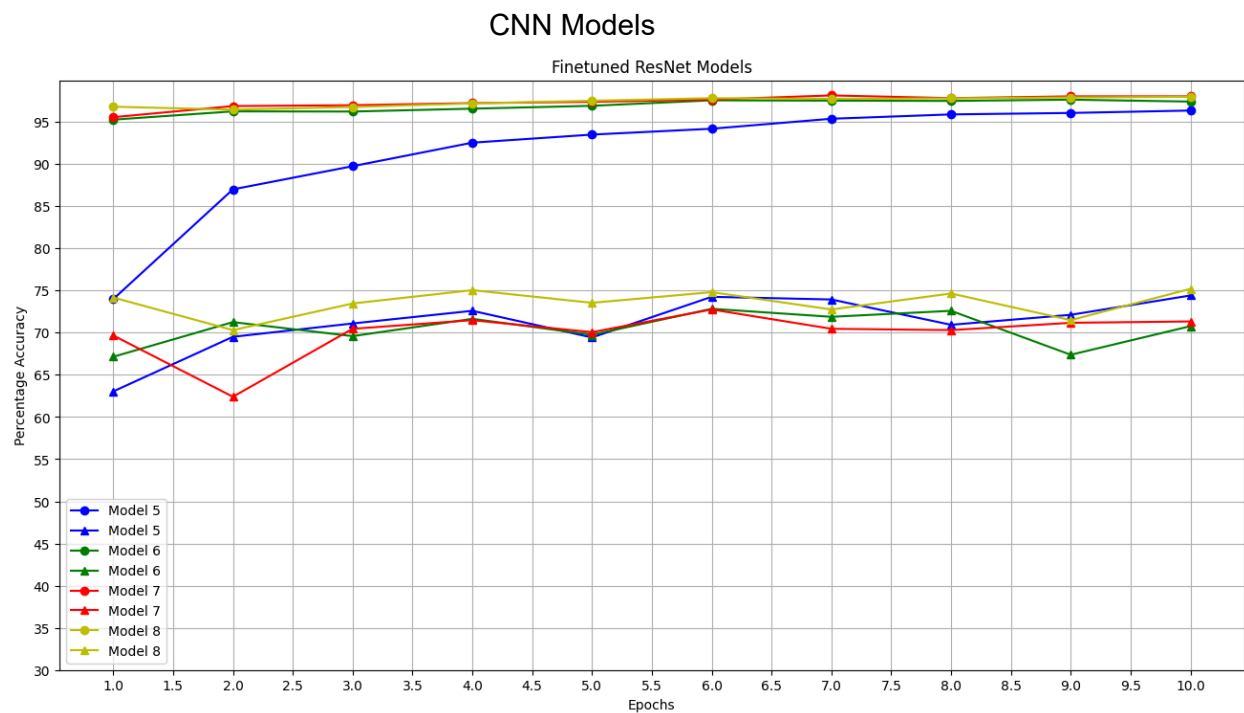
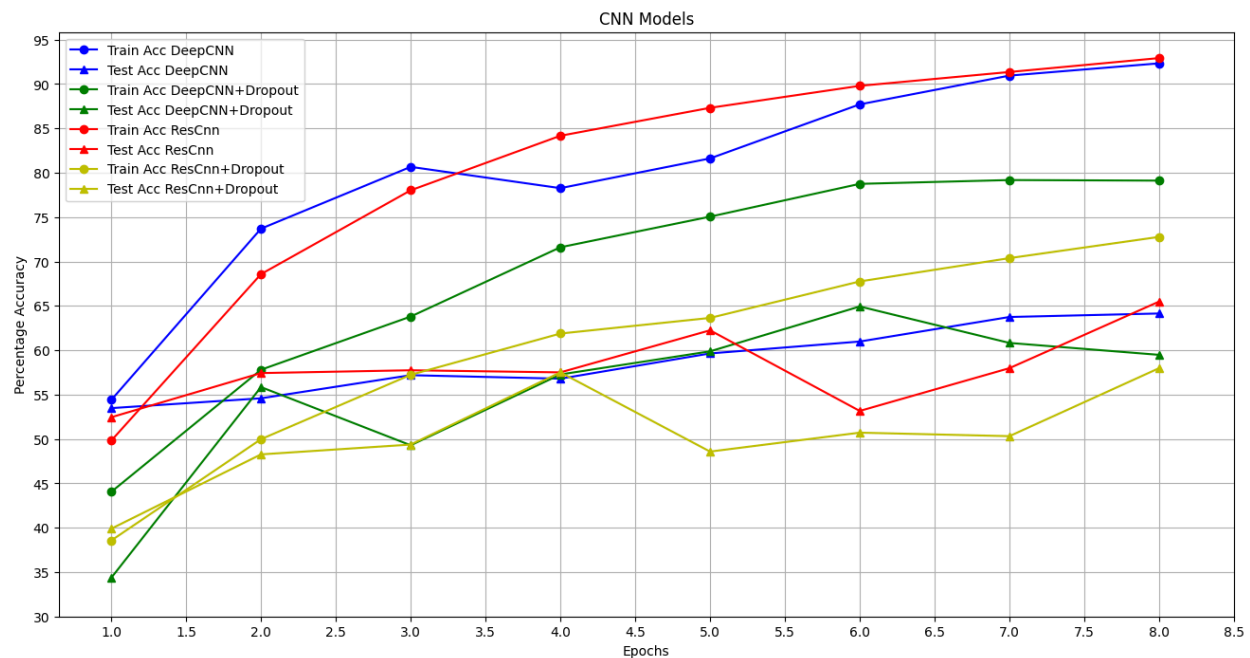


OncoNet WiDS 5.0

CNN vs ResNet Performance for ISIC Data



ResNet FineTuned Models

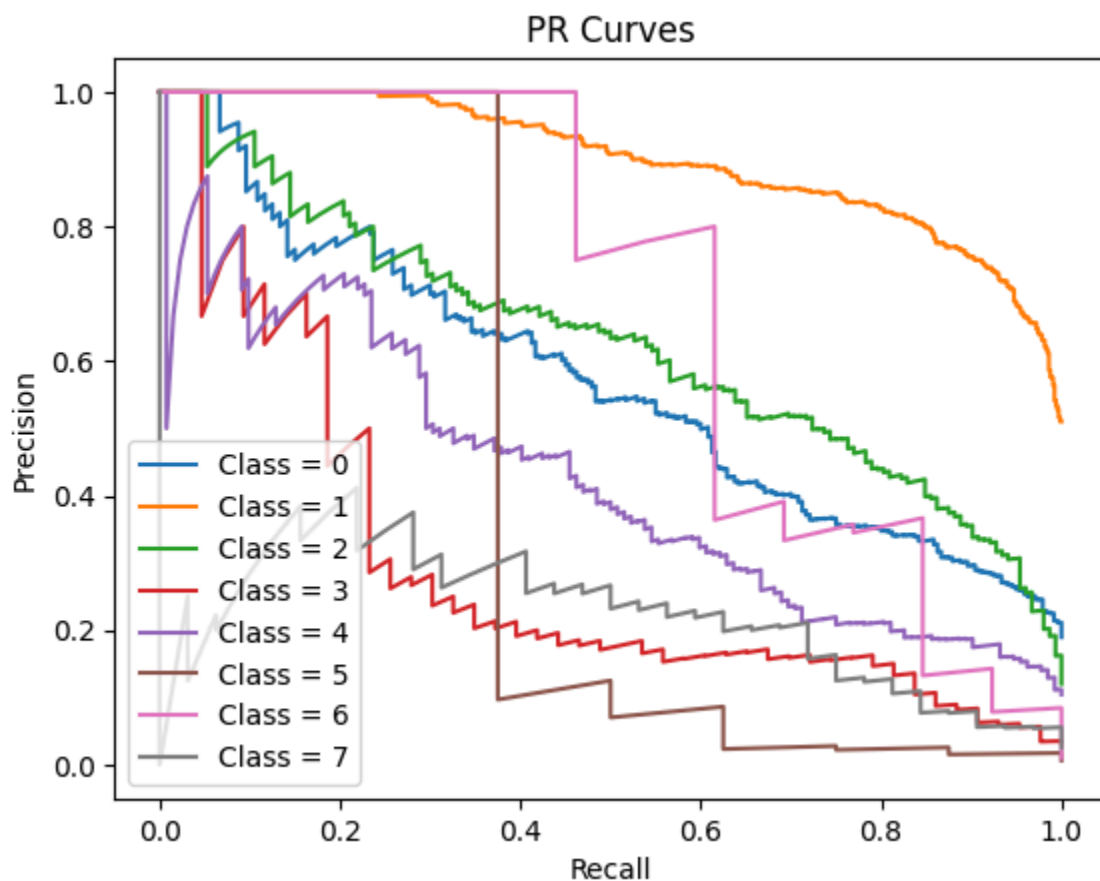
As we can see above, the best pre-trained ResNet18 model (Model 8) achieved 90+% training accuracy in the first epoch itself, while the scratch CNN models could achieve that good training accuracy only after 7-8 epochs

The ResNet18 model being deeper, the finetuned models could capture more features, and hence it resulted in better training and testing accuracy than that of CNN models

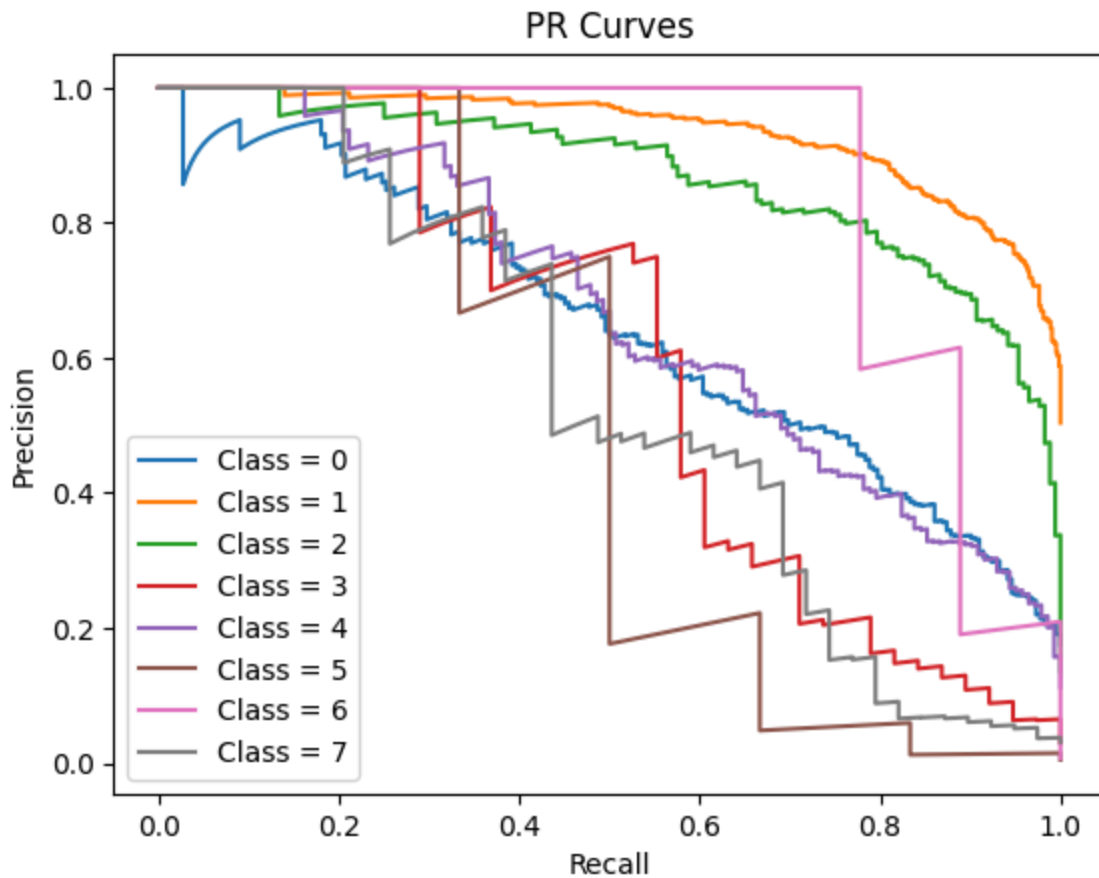
The precision, recall, F1 score, and weighted accuracy of model 8 are about 10% better than those of CNN models. Even the PR curves and confusion matrix indicate that Finetuned models could learn better. The precision, recall, and F1 score being close to the raw accuracy indicate that the model could learn properly, even in this case of biased multi-class classification. This is because of the relative proportion of images taken in the training data.

Summary:

Shallow CNN Models reached a peak accuracy of 65% while pre-trained ResNet deep models with fine-tuning achieved a peak accuracy of 76%.



PR Curve for CNN Model



PR curve for fine tuned resnet18 model

Implementation of Best CNN Architecture:

```
model=nn.Sequential(
    nn.Conv2d(3,32,3,padding=1),
    nn.ReLU(),
    nn.BatchNorm2d(32),
    nn.Conv2d(32,32,3,padding=1),
    nn.ReLU(),
    nn.BatchNorm2d(32),
    nn.MaxPool2d(2,2),
    # nn.Dropout(0.3),
    nn.Conv2d(32,64,3,padding=1),
    nn.ReLU(),
    nn.BatchNorm2d(64),
    nn.Conv2d(64,64,3,padding=1),
    nn.ReLU(),
    nn.BatchNorm2d(64),
    nn.MaxPool2d(2,2),
    # nn.Dropout(0.3),
    nn.Conv2d(64,128,3,padding=1),
    nn.ReLU(),
```

```

nn.BatchNorm2d(128),
nn.Conv2d(128,128,3,padding=1),
nn.ReLU(),
nn.BatchNorm2d(128),
nn.MaxPool2d(2,2),
# nn.Dropout(0.4),
nn.Flatten(),
nn.Linear(12*12*128,256),
nn.ReLU(),
nn.BatchNorm1d(256),
nn.Linear(256,128),
nn.ReLU(),
nn.BatchNorm1d(128),
# nn.Dropout(0.3),
nn.Linear(128,8)
)

```

As we can see the above model is a deep cnn model with batch normalization , relu activation function , pooling and dropouts

Implementation of Residual CNN Architecture:

```

model=nn.Sequential(
    nn.Conv2d(3,32,3,padding=1),
    Residual(conv_block(32,32)),
    nn.MaxPool2d(2,2),
    # nn.Dropout(0.3),
    nn.Conv2d(32,64,3,padding=1),
    Residual(conv_block(64,64)),
    nn.MaxPool2d(2,2),
    # nn.Dropout(0.3),
    # Residual(conv_block(128,128)),
    nn.MaxPool2d(2,2),
    # nn.Dropout(0.3),
    nn.Flatten(),
    nn.Linear(12*12*64,256),
    nn.ReLU(),
    nn.BatchNorm1d(256),
    nn.Linear(256,128),
    nn.ReLU(),
    nn.BatchNorm1d(128),
    nn.Linear(128,8),
    nn.Flatten(),
    nn.Linear(12*12*64,256),
    nn.ReLU(),
    nn.BatchNorm1d(256),
    nn.Linear(256,128),
    nn.ReLU(),
    nn.BatchNorm1d(128),
    nn.Linear(128,8))

```

Fine Tuning of ResNet architecture was done , it's architecture is given by,

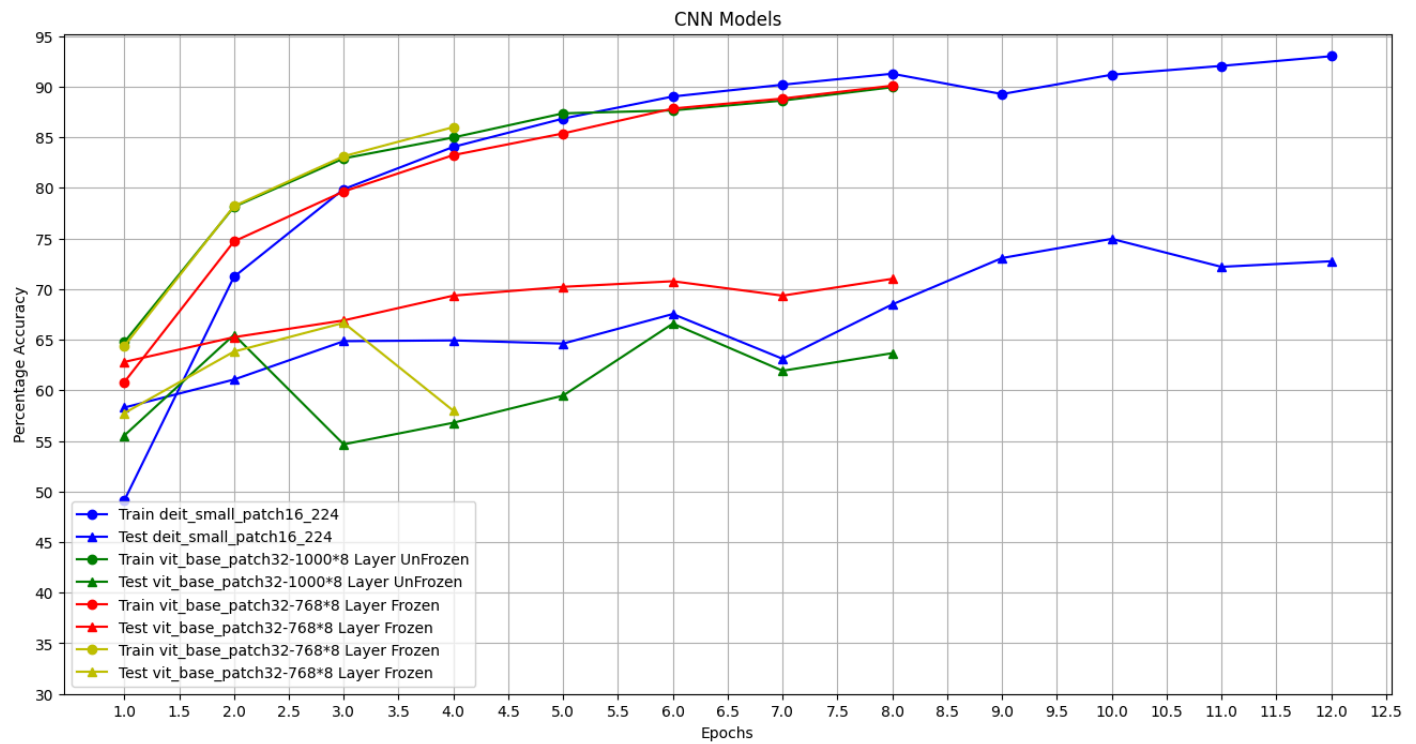
```
model=nn.Sequential(Resnet18_pretrainedmodel,
                    nn.Linear(1000,128),
                    nn.BatchNorm1d(128),
                    nn.ReLU(),
                    nn.Dropout(0.5),
                    nn.Linear(128,128),
                    nn.BatchNorm1d(128),
                    nn.ReLU(),
                    nn.Dropout(0.5),
                    nn.Linear(128,8),
                    )
```

Vision Transformers:

The training loop and testing loop is given below:

```
train_model(model,train_loader,test_loader,optimizer,loss_fn,train_accuracy,test_accuracy,epochs=5)
for epoch in range(epochs):
    model.train()
    total_loss=0
    total=0
    correct=0
    step=0
    for x,y in train_loader:
        step+=1
        x,y=x.to(device),y.to(device)
        optimizer.zero_grad()
        logits=model(x)
        loss=loss_fn(logits,y)
        loss.backward()
        optimizer.step()
        total_loss+=loss.item()
        pred=logits.argmax(dim=1)
        correct+=(pred==y).sum().item()
        total+=y.size(0)
    if step%20==0:
        print(f"Step {step} : Intermediate Accuracy = {100*correct/total :.2f}")
    print(f"Epoch {epoch+1}")
    print(f"Loss : {total_loss:.4f}")
    print("Training Accuracy")
    train_accuracy.append(100*correct/total)
    print(f"Accuracy: {100*correct/total :.2f}")
    print("Testing Accuracy")
    test_accuracy.append(test_model(model,test_loader))
```

The Comparison for various Vision transformers is given below:

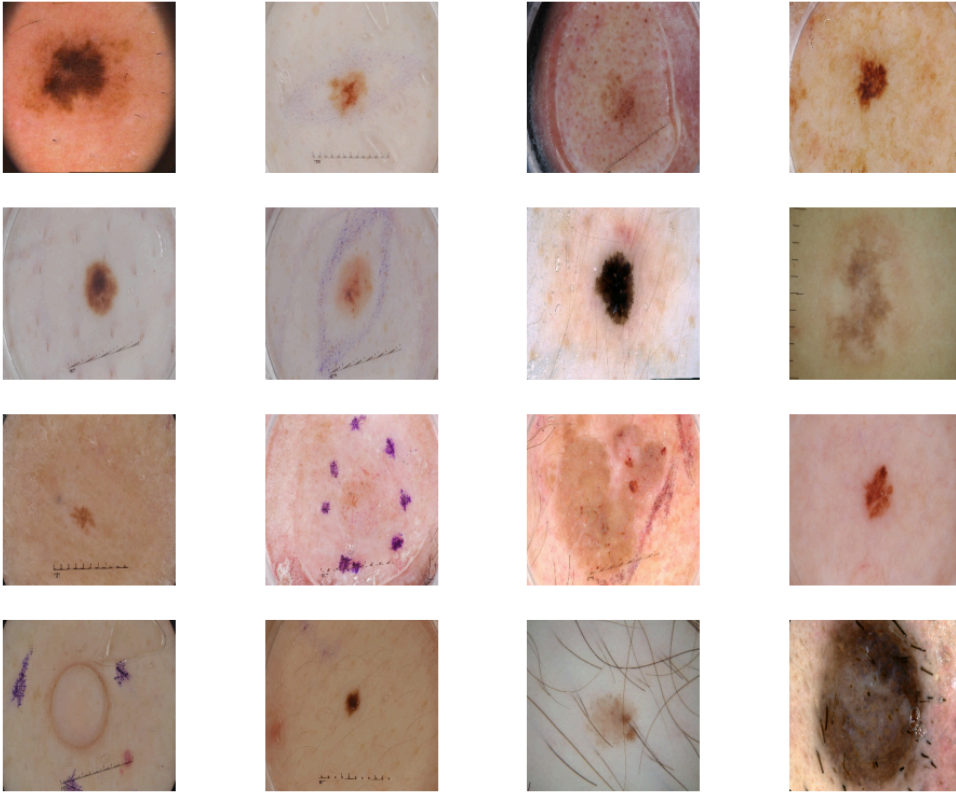


As we can train deit_small_patch transformer is performing the best

Lesion Segmentation:

Example:

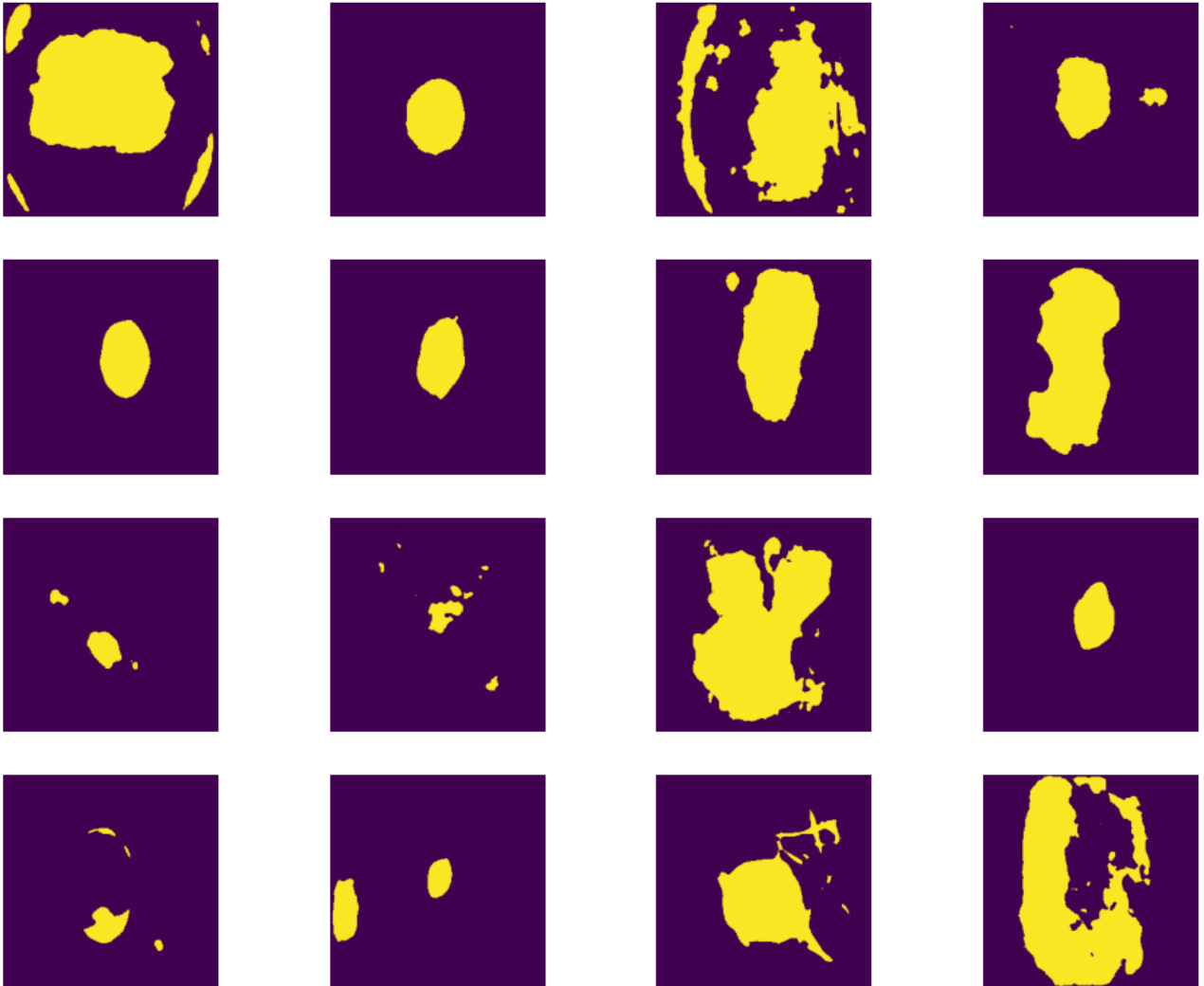
The below image contains original segments



The segment pictures of the above images are



The lesion segmentation of the trained model are



```

def dice_coeff_batch(pred, target, smooth=1):
    pred = pred.view(pred.size(0), -1)
    target = target.view(target.size(0), -1)
    intersection = (pred * target).sum(dim=1)
    total = pred.sum(dim=1) + target.sum(dim=1)
    dice_per_sample = (2. * intersection + smooth) / (total + smooth)
    return dice_per_sample.sum()

def iou_coeff_batch(y_true, y_pred, eps=1e-7):
    intersection = (y_true * y_pred).sum(dim=(1,2,3))
    union = y_true.sum(dim=(1,2,3)) + y_pred.sum(dim=(1,2,3)) - intersection
    iou = (intersection + eps) / (union + eps)
    return iou.sum().item()

def iou_score(preds, targets, eps=1e-7):
    preds = preds.float()
    targets = targets.float()
    intersection = (preds * targets).sum()
    union = preds.sum() + targets.sum() - intersection
    iou = (intersection + eps) / (union + eps)
    return iou.item()

```

These above are the stats calculated for segmentation model comparisons

```

class Residual(nn.Module):
    def __init__(self, fn):
        super().__init__()
        self.fn = fn
    def forward(self, x):
        return nn.functional.relu(self.fn(x) + x)

def conv_block(input_channel, output_channel, residual=False):
    if not residual:
        return nn.Sequential(
            nn.Conv2d(input_channel, output_channel, 3, padding=1),
            nn.BatchNorm2d(output_channel),
            nn.ReLU(),
            nn.Conv2d(output_channel, output_channel, 3, padding=1),
            nn.BatchNorm2d(output_channel),
            nn.ReLU()
        )
    else:
        return nn.Sequential(
            nn.Conv2d(input_channel, output_channel, 3, padding=1),
            nn.BatchNorm2d(output_channel),
            nn.ReLU(),
            nn.Conv2d(output_channel, output_channel, 3, padding=1),
            nn.BatchNorm2d(output_channel)
        )

```

The above architecture is the res net architecture

```
class UNET(nn.Module):
    def __init__(self, in_channels=3, out_channels=1, features=[64, 128, 256, 512],):
        super(UNET, self).__init__()
        self.ups = nn.ModuleList()
        self.downs = nn.ModuleList()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        for feature in features:
            self.downs.append(conv_block(in_channels, feature))
            in_channels = feature

        for feature in reversed(features):
            self.ups.append(nn.ConvTranspose2d(feature*2, feature, kernel_size=2, stride=2,))
            self.ups.append(conv_block(feature*2, feature))

        self.bottleneck = nn.Conv2d(features[-1], features[-1]*2, 3, padding=1)
        self.final_conv = nn.Conv2d(features[0], out_channels, kernel_size=1)

    def forward(self, x):
        skip_connections = []

        for down in self.downs:
            x = down(x)
            skip_connections.append(x)
            x = self.pool(x)
        x = self.bottleneck(x)
        skip_connections = skip_connections[:-1]
        for idx in range(0, len(self.ups), 2):
            x = self.ups[idx](x)
            skip_connection = skip_connections[idx//2]
            if x.shape != skip_connection.shape:
                x = TF.resize(x, size=skip_connection.shape[2:])
            concat_skip = torch.cat((skip_connection, x), dim=1)
            x = self.ups[idx+1](concat_skip)
        x=self.final_conv(x)
        return x
```

The above architecture is the UNET Architecture